

让团队资产的作用看得见

技术实现、参数依据与真实模型案例

GitHub · 项目已开源

github.com/yuwxyun275/team-assets-memory-hub



形成经验	新任务采用	核验结果
从开发记录中提炼经验 审核后发布为团队资产	CodeBuddy 读取已发布经验 按经验中的方法检查库存功能	按选定方法执行测试并通过 独立验证器再次测试并通过

贯穿案例：单进程库存请求幂等性

从重复扣减缺陷出发，展示一份经验怎样产生、被理解和使用，以及如何生成下一份修订候选。全文沿竞赛六项任务展开，配有当前界面的真实截图与实现说明。

这个案例展示经验如何被实际使用，以及结果如何核验。是否提高任务完成率或节省成本，在第五章结合对比实验单独讨论。

技术实现与案例报告 | TencentDB Agent Memory

依据：公开源码与可复现评测：github.com/yuwxyun275/team-assets-memory-hub。

导读 | 阅读路线与案例背景

0.1 先看主线，再看评测分支

六项竞赛任务共同构成闭环。任务五回答“是否带来增益”，可以独立开展，不阻塞每一次开发。



05 效果评测是独立分支：冻结任务与资产 → 对照执行 → 计算收益、成本和不确定性。它读取主线证据，不作为每次任务继续执行的前置条件。

主线中，上一阶段的输出就是下一阶段的输入。资料形成候选，审核后的版本进入检索。Agent 读取并采取行动后，系统记录结果、生成回执，再决定是否提出修订或新候选。

章节	页码
任务一 历史开发经验与项目知识学习	5-11
任务二 面向新任务的资产检索与最小上下文组织	12-14
任务三 资产使用链路记录与可信归因	15-16
任务四 用户可感知的资产使用说明	17-20
任务五 资产效果评测与反事实比较	21-24
任务六 任务经验回流与候选资产生成	25-28

报告以库存案例展示资料如何被提炼为资产、经过审核发布，再被 CodeBuddy 使用，并在任务结束后生成修订建议。第五章另外介绍参数的选择依据和对比实验结果。

依据：竞赛题目四。参数实验记录见 [evaluation/parameter_calibration/](#)。

导读 | 阅读路线与案例背景

0.2 贯穿案例：重复请求不能重复扣库存

先修复，再复用。任务 A 负责产生历史经验，任务 B 负责验证已发布经验是否参与后续工作。

阶段	任务与输入	真实输出
任务 A	单进程 Python 库存示例。相同 request_id 重复调用，库存实际变为 8，预期为 9	修复代码。3 项测试通过。留下失败、修复和回归轨迹
形成经验	将任务 A 的工具记录、代码、测试和约定提交学习	生成项目经验候选。修订、审核并经负责人批准发布
任务 B	检查已有库存实现，运行回归。仅发现问题时改代码	读取经验，采用验证方法。5 项测试及独立本地验证通过
继续回流	对照任务 B 的结果和已发布版本	提出修订候选，保留证据，等待审核



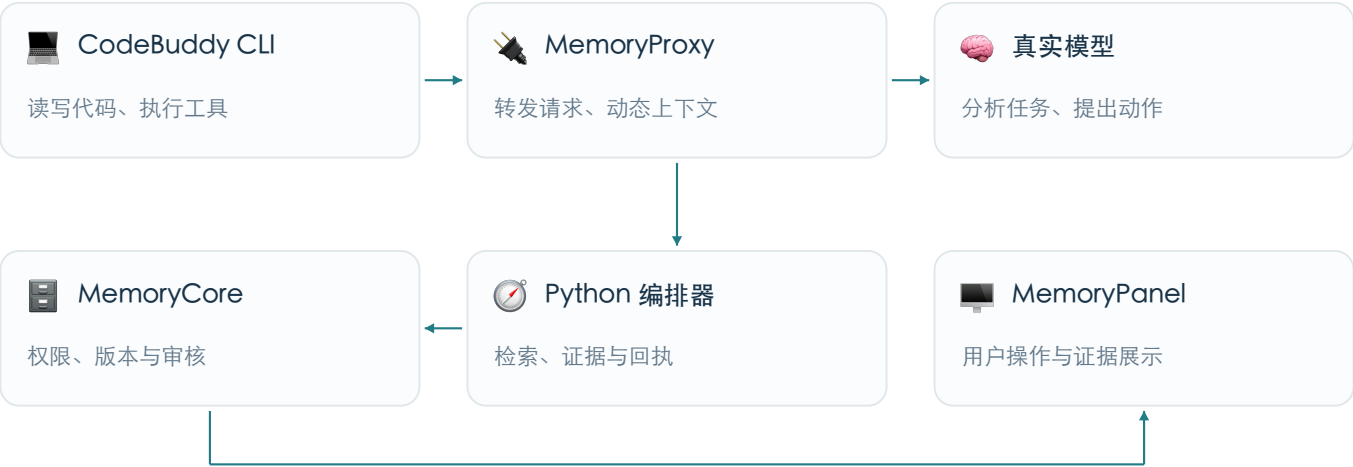
图 1 | 真实体验团队中的任务 A 与任务 B。看板“完成”指工程结果，不直接表示资产有因果贡献。

库存工程和约定是示例资料。模型回复、工具调用、测试与提炼真实发生。任务 A 与 B 的目标不同，不能拿它们相减得出资产收益。

依据：real-run-summary.json。real-ci-1.json。real-reuse-receipt.json。截图 01。

0.3 组件怎样配合完成一次任务

请求通路负责让 Agent 工作。资产与证据通路负责提供受控上下文，并把观察保存下来。



上层：请求通路。下层：资产、证据与界面。箭头表示主要调用或数据方向。

组件	承担的责任	不替代什么
CodeBuddy CLI	理解任务、读写代码、调用工具和测试	自己的采用声明不等于独立证明
MemoryProxy	绑定身份与任务，提供资产卡片，采集送达后的行为	不绕过 Core 的权限与发布约束
编排器与 Core	检索与预算选择。内容快照、审核、来源和使用账本	质量分不代替测试， 测试不代替收益对照
MemoryPanel	展示候选、来源、各轮回执和待审核内容	页面统计不会自动扩大证据结论

下文沿实际业务顺序展开。每章最后说明产物交给谁，技术参数在出现用途之后解释。

依据：MemoryProxy/src/assets/。MemoryCore/src/asset-quality/。evaluation/team_asset_bench/。MemoryPanel/web/src/pages/。

任务一 | 历史开发经验与项目知识学习

1.1 先让一次开发留下可学习的材料

任务 A 尚未使用团队资产。它产生的执行轨迹，才是后续经验的来源。



任务一 | 历史开发经验与项目知识学习

1.2 提交资料：先说明属于哪个项目

项目身份、版本和适用范围用于防止经验被错用。正文可以是原始开发会话、文档、代码或产品知识。

从原始资料学习

从这里开始

提供项目资料，系统帮你整理可复用的经验。你只需说明资料属于哪个项目、适用于什么场景。

1 你提供资料
填写下方表单

→

2 系统提炼候选
也可能建议复用已有资产

→

3 负责人审核发布
审核通过后用于团队任务

资料属于哪个项目？

标有「必填」的内容都需要填写

项目 / 仓库路径 必填

项目版本 必填

例如：/Users/yourname/projects/inventory

例如：v1 或 v1.2.0

填写运行 CodeBuddy 的项目根目录。在项目终端输入 `pwd`，复制输出的完整路径；后续任务请使用同一路径。

填写版本号或 Git 提交号。体验示例用 `v1`；请与后续任务的版本保持一致。

这些资料适用于什么场景？ 必填

例如：库存模块的重复请求处理和回归测试；活用于单进程，不适用于多实例并发。

图 2 | 项目、版本和范围的填写提示。表单为空，展示实际入口。

仅本人

☒

先整理个人资料，限制其他成员访问

当前团队

☐

允许本团队成员查看，发布仍需审核

提交后会怎样？

系统保留来源并提炼候选。生成的内容需经负责人审核，才会发布为团队资产。

提交资料并提炼候选

还需填写：项目路径、项目版本、适用范围、原始资料。

图 3 | 选择资料与候选的可见范围。普通上传默认不是模拟资料。

当前支持 TXT、Markdown、JSONL 和代码文本。JSONL 作为会话资料接收，尚不等于兼容每一种 CLI 导出的角色与工具结构。单份最多 60,000 字符，最多 24 份，批次最多 140,000 UTF-8 字节。

本例从真实 CLI 记录选取相关工具事件，并附代码、测试和约定。来源权限和内容真实性仍需核对，上传按钮本身不是认证。

依据：LearningPanel.tsx。learning-service.ts。walkthrough-learn-session.mjs。截图 02、03。

任务一 · 小节 1.2 / 本章 7 节

6 / 28

任务一 | 历史开发经验与项目知识学习

1.4 先认识 Q、E、U，再讨论能否发布

三个指标回答不同问题。首次发布需要内容审核和负责人批准，U 可以暂时未知。

指标	回答的问题	依据与阶段
Q · 内容质量	正文是否正确、完整、有边界、可使用	模型逐项给出 0-4 分与理由。程序按维度汇总
E · 证据覆盖	审阅结论能否回到材料核对	有明确结论且有引用的检查项占比。负面证据也算覆盖
U · 使用观察	当前版本是否在该情境中观察到帮助	使用后的行为与反馈。同任务去重，见任务四

当前 Q：四维等权，避免无依据的权重差异

Q = 四个维度分数的平均值，再四舍五入

E = 有结论且有引用的模型检查项 / 模型检查项总数

正确性、完整性、边界性、可用性各占 25%。这是对四维同等重视的明确策略，不宣称它是实验拟合出的最优权重。缺分或缺证据时 Q 保持未知。



全局默认门槛仍为 80。E 不是合格率，Q 不是正确概率。历史快照保留当时评分，不被新公式改写。参数复查见 5.3 节。

依据：scorecard.ts。decision-policy.ts。lifecycle.ts。docs/PARAMETER_EVIDENCE_CN.md。

任务一 | 历史开发经验与项目知识学习

1.5 核对历史审核：把分数与原文放在一起

这张截图记录的是库存经验当时的发布审核。它使用旧版评分规则，属于保留的历史证据。

▼ 查看自动评估详情

内容质量：90 / 100 · 证据覆盖：100% · 发布门槛：75，且必需检查全部通过

内容标识：learned-64232c6e8008f710bd6d993cd9d96294 · 内容版本：1cb3288ee1039063987d36d14d1e59be037dbd865fa52155001e6a6f5837dacd · 源资产版本：1

评估版本：507bf1bc48e3471b2aa699789b7c91d4c85579bcdccb237568eb7c1a8b093a9e · 尝试：1/3

▶ 明显凭据检查 · 通过 · —/4

▶ 实际正文 · 通过 · —/4

▶ 声明用途 · 通过 · —/4

▶ 可核对的配套材料 · 通过 · —/4

▶ 适用前提与边界 · 通过 · 4/4

▶ 正文完整性与一致性 · 通过 · 3/4

▶ 安全与风险边界 · 通过 · 4/4

▶ 事实与来源核对 · 通过 · 4/4

▶ 知识可复用性 · 通过 · 3/4

图 5 | 历史发布版本 Q=90、E=100%。当时各项检查通过，体验团队的门槛经授权设为 75。

相同四维分数	历史 v2	当前 v3 的演算示例
100、75、100、75	40%、25%、20%、15% → Q=90	四维各 25% → 87.5，取整 Q=88
5 项均有结论和引用	E=100%	E 定义保持不变

右栏只说明新公式如何计算，不是重新审批这份资产。原审核时间、报告、人工批准和当时门槛都保留。

依据：real-publication.json。截图 06。intrinsic-quality/v2 历史记录与当前 v3 规则。

任务一 | 历史开发经验与项目知识学习

1.6 负责人发布：确认这份内容可被复用

候选已经有正文和来源，无需再填写一份空白“内容单元”。负责人审核的是这份固定版本。

候选与已发布资产

选择一项资产，查看内容、来源、审核结果和使用记录。

选择资产

库存示例幂等修复的验证闭环：先复现失败再修复再全量回归 · llm_wiki · approved

候选内容与审核发布

先核对正文、来源和适用范围，再由负责人审核发布。已有候选无需重新填写或提交。

库存示例幂等修复的验证闭环：先复现失败再修复再全量回归

已发布

此版本已发布，可用于团队任务。后续采用和验证情况见下方使用记录。

1. 核对内容与来源

适用范围

仅用于 inventory-real @ v1 的单进程库存示例及其现有三个 pytest 用例。并发、多实例、重启后去重、持久化及生产适用性均未验证。

适用项目： /Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/inventory-real · 项目版本： v1 · 合成资料

图 6 | 已发布的库存项目经验。资产类型为 llm_wiki，正文说明请求去重的验证方法。

本例用户批准的范围仅为本地单进程库存体验。修改正文、来源或适用范围，会生成新的审核快照，不直接覆盖已经批准的版本。

输出 → 已批准的资产版本。检索模块只把符合当前权限、版本与范围的内容交给后续任务。

依据：real-publication.json。lifecycle.ts。截图 04。

任务一 | 历史开发经验与项目知识学习

1.7 来源链：每个结论都能返回材料

资产保留的不只是一个标题，还包括来源位置、版本、内容指纹和支持引用。



图 7 | 历史审核版本保留了真实工具记录、代码、测试、产品约定和关联候选。

保存内容	作用
原文与引用位置	核对结论是否得到材料支持
版本与内容指纹	确认后续读取的是同一份内容
权限和适用范围	限制谁可以读，以及在哪些任务里使用

来源里的“测试通过”文字与系统独立执行的测试是不同证据。SHA-256 固定内容，不认证来源真实性。下一章开始处理任务 B。

依据：learning-evidence.ts。real-session-learning-input-1.json。截图 05。



任务二 | 面向新任务的资产检索与最小上下文组织

2.1 任务 B 开始：先确认适用，再组织上下文

任务 B 要核对现有库存实现并运行回归。它需要适用的验证经验，不需要整库历史材料。



输入	本例的含义
任务目标	检查库存请求幂等性。发现问题才修改代码
项目环境	inventory-real @ v1。单进程 Python 示例
候选资格	当前身份可读，版本匹配，已完成发布审核
上下文预算	由任务画像与服务配置限制条数和 Token

当前实现把未发布、已纠错、废弃、错版本或绑定错仓库的资产提前排除。它们既不能注入，也不能挤占召回窗口或改变有效语料的 IDF。

输出 → 当前任务可选的资产集合。下一节解释这些资产之间怎样排序。

依据：orchestrator.py、native_assets.py、task_profile.py。权限仍由 Core 数据面核验。

任务二 | 面向新任务的资产检索与最小上下文组织

2.2 排序参数：保留有用途的量，移除重复加分

旧版先融合检索分数，再把 RRF 和九类信号加权。当前方案使用一次 RRF，资格条件交给门禁。

$RRF(asset) = \sum 1 / (k + \text{该资产在每路检索中的名次})$
当前 $k=10$ 。三路同权。缺失的名次不计入。
 U_{eff} = 将 U 与适用性调整合并后限制在 0 到 1
最终分 = 归一化 $RRF \times [1 + 0.05 \times (2U_{eff} - 1)]$

设置	当前值	依据
RRF 常数 k	10	在冻结的开发任务上比较 10、30、60、100 后选出
候选窗口	24	比较 12、24、48，按覆盖、冗余、成本顺序选择
相对分数底线	0.2	比较 0.2、0.4、0.6，只保留达到本轮最高分一定比例的项
Q、版本、权限	作为资格条件	不再次制造“可信度 15%、版本 10%”等相关性分数
U 的影响范围	相对分数最多 ± 5%	显式控制反馈影响，不是经验最优值

RRF 方法来自检索研究。论文使用的 $k=60$ 是本次候选之一，不能直接当成本项目最优值。完整比较和保留集结果放在 5.2 节。

这套默认值有本地契约任务上的依据，适用范围也有记录。它不保证所有仓库、语言或向量模型下最优。

依据：decision_policy.py。hybrid_retrieval.py。evaluation/parameter_calibration/results/ranking.json。Cormack 等，SIGIR 2009。

任务二 | 面向新任务的资产检索与最小上下文组织

2.3 交付上下文：用户能看到推荐，模型按需读取

卡片提供名称、版本、用途和读取入口。正文真正送达后，系统才进入注入与使用观察。



图 8 | 库存真实任务的画像与预算界面。该截图保留当时运行配置，不是本次参数实验的 6,000 Token 设置。

同一起来源和版本的明确内容身份会用于去重。条数上限与 Token 预算容量约束，并不保证“刚好够用”。超出预算的内容不注入，任务也可以完全不选资产。

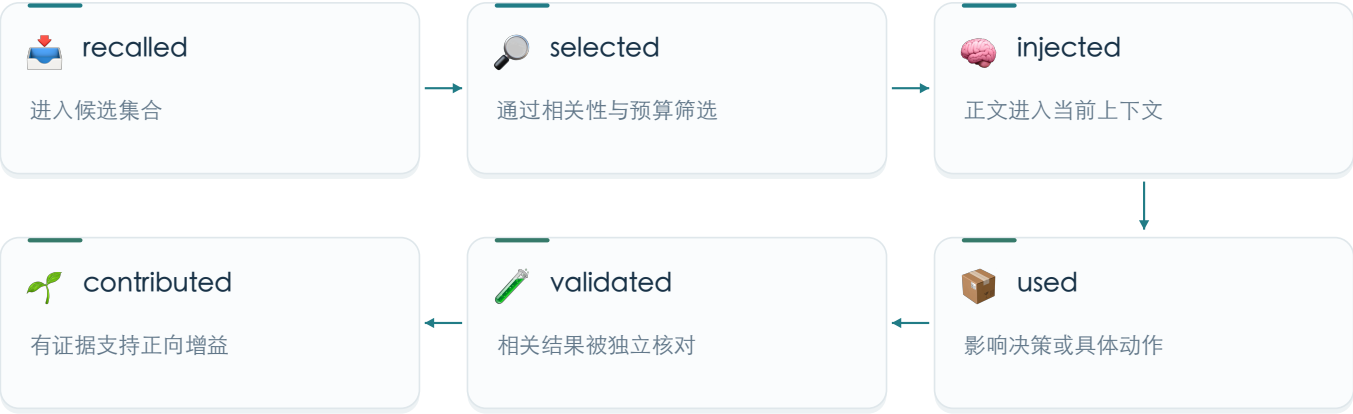
输出 → 选中的资产与送达记录。被推荐或读到仍不等于被采用，下一章检查实际行为。

依据：MemoryProxy 注入与读取链路。orchestrator.py。截图 11。

任务三 | 资产使用链路记录与可信归因

3.1 证据链：每进一步，都要新增依据

系统区分资产出现在哪里、Agent 做了什么，以及结果是否验证。不能用前一阶段代替后一阶段。



关联	最低需要看到什么
asset → decision/change	资产建议与代码区域、工具调用、测试或风险检查对应
decision/change → outcome	实际命令、退出码、测试结果或独立验证记录
outcome → contribution	可归因的对照或独立验证结论，不能只看到任务成功
corrected	用户或执行结果指出错误、过期或不适用，保留纠正依据

采用声明用于定位关联，必须与观察到的操作相互核对。生成一份“测试通过”的报告文件，不能替代真实测试。

输出 → 资产、动作和结果之间的可追踪关联。下面核对任务 B 的同一轮证据。

依据：ledger.py、attribution.py、contribution.py。team-assets-evidence-observer.ts。

任务三 | 资产使用链路记录与可信归因

3.2 任务 B 第 2 轮：经验影响了验证动作

本例的采用对象是验证方法。任务 B 不必再制造一次实现修改，才能证明经验参与了工作。



图 9 | 展开任务 B 第 2 轮。记录“影响了操作”与“已被验证”，并保留独立 local-ci 结果。

经验建议	本轮可核对的行为
核对接口和 seen 的单进程范围	读取现有实现与测试后检查条件
同 ID、不同 ID、交错请求三类场景	采用声明指向三个对应的测试目标
执行回归并保存结果	实际 pytest 输出 5 passed。独立本地验证再次通过

实际测试：python3 -m pytest -v test_inventory.py
工具输出：5 passed，退出码 0
独立 local-ci：再次执行，5 项通过

这里固定看第 2 轮。后续仅复核的轮次没有重复登记采用。早期观察器字段曾不一致，原始记录保留，核验以工具输出与独立测试为准。

依据：real-reuse-receipt.json。real-ci-reuse.json。real-evidence-reconciliation.json。截图 10。



任务四 | 用户可感知的资产使用说明

4.1 回执先回答用户的问题，再允许展开证据

用户首先需要知道用了什么、影响哪里、验证到什么程度，而不是阅读一串内部事件名。



用户的问题	回执提供的答案
本次用了什么	具体资产名称、类型、内容版本和来源
为什么适用	当前问题、仓库、环境与资产条件的对应
影响了什么	决策、代码位置、工具调用或测试方法
结果可信到哪一步	区分观察性帮助、实际验证与因果贡献
还需要我做什么	发布需负责人审核。使用反馈用于纠错，不要求每次填写

自动评价读取“推荐之前”的情境与“送达之后”的行为。它说明适用性和观察到的帮助，不替代测试，也不替用户作业务验收决定。

输出 → 可阅读的资产使用说明。下面分别查看使用后的证据和推荐前的条件，避免把时间顺序混在一起。

依据：UsageEffectReceipts.tsx。usage-reviewer.ts。usage-result.ts。

任务四 | 用户可感知的资产使用说明

4.2 使用后的评价：帮助必须对应实际行为

后台评价不是让模型随手打分。它要引用资产里的具体建议，并找到后续实际行为的对应证据。

② 正文送达后的实际效果

模型评价：有帮助

资产建议核对当前函数仍采用三个参数、测试使用同一进程内 seen 集合，并运行 pytest 检查三类行为。后续工具读取 inventory.py 显示函数签名与资产描述一致（def reserve(stock, request_id, seen)），测试文件包含资产提到的三个测试（test_duplicate_request、test_distinct_request、test_duplicate_request_interleaved_with_distinct），且实际运行 pytest 输出 5 passed、PYTEST_EXIT=0，其中三个测试通过。这构成具体实现与对应工具结果的对应关系，可观察采纳帮助。但无法排除其他信息来源，且资产仅覆盖 3 个用例，当前仓库有 5 个用例，后两个超出资产范围。

对应的当前资产原文

先核对当前函数仍采用上述三个参数，测试仍使用同一个进程内的 `seen` 集合。接口或约定变化时停止套用，重新确认测试要求。

证据 1484ec7fc9c429f201f7e62b5b4efcaaf4c9aa370ce84e724022cd2a975ffc5d · tool_result · 工具调用
call_00_71miWm2dZk3tOb8z1EXs3002

1→def reserve(stock, request_id, seen):

证据 d2db305dc1272e7f7fadb70b31cab5f3099de665aa171d320ee034b94e8e14a · tool_result · 工具调用
call_00_dTSpBkeYA2QptwsftMHC6680

3→def test_duplicate_request():

证据 d2db305dc1272e7f7fadb70b31cab5f3099de665aa171d320ee034b94e8e14a · tool_result · 工具调用
call_00_dTSpBkeYA2QptwsftMHC6680

8→def test_distinct_request():

证据 d2db305dc1272e7f7fadb70b31cab5f3099de665aa171d320ee034b94e8e14a · tool_result · 工具调用
call_00_dTSpBkeYA2QptwsftMHC6680

13→def test_duplicate_request_interleaved_with_distinct():

证据 3ec753b057600bae6a99225afcb13805ce9e1d08b3516326a29d331a54994f9c · tool_result · 工具调用
call_00_RKZIHkYKvc50vgZ8d4OAc0461

test_inventory.py::test_duplicate_request PASSED [20%]

图 10 | 真实异步评价中的资产原文和事件引用。它来自第 1 轮推荐后持续观察的窗口，与第 2 轮采用账本分别标识。

模型指出接口、seen 的范围和测试场景与经验一致，并引用真实 pytest 输出。它仍保留“无法排除其他信息来源”的限制。helpful 是观察性评价，不直接升级为 contributed。

本例历史观察曾汇总为 $U \approx 52.4$ 。当前规则在 4.4 节说明，历史记录与截图不被改画成新值。

依据：usage-reviewer.ts、usage-result.ts。历史异步观察窗口。截图 08。

任务四 | 用户可感知的资产使用说明

4.3 推荐前的判断：只用当时已经知道的条件

判断“当时为什么推荐”，必须使用推荐前冻结的内容。不能看到成功结果后，再倒推当时一定合适。

资产使用效果回执（异步评估）

推荐前的条件用于判断适用性，正文送达后的行为用于判断效果。两项分别评估，不替代实际采用、独立测试和贡献证明。

资产版本 1 · 去重观察 2 · 已评价 2 · 有帮助 2 · 待人工 0

库存示例幂等修复的验证闭环：先复现失败再修复再全量回归

· 评估完成

① 推荐前的场景 · 第 1 轮

请回归检查 inventory.py 的请求幂等性：重复请求不能重复扣减，不同请求需正常扣减。按实际需要查阅已发布的团队经验，核对当前实现并运行测试，报告实际证据和未覆盖边界。仅在发现问题时修改代码。

当时是否适用：当时适用

用户要求回归检查 inventory.py 的请求幂等性，重复请求不能重复扣减，不同请求需正常扣减，并运行测试。资产明确适用于 inventory-real v1 的 reserve(stock, request_id, seen) 及配套测试，且提供了相同的检查场景（同一请求重试、两个不同请求、交错请求）和测试命令。用户要求与资产适用范围一致，无冲突条件。

资产条件：仅用于本体验项目 `inventory-real`、版本 `v1` 中的 `reserve(stock, request\_id, seen)` 及配套测试。

推荐前证据 · query-579386c40919fbe1c873e1ce924ea53b0707740d23a50d9912347dc88e637da7

请回归检查 inventory.py 的请求幂等性：重复请求不能重复扣减，不同请求需正常扣减。按实际需要查阅已发布的团队经验，核对当前实现并运行测试，报告实际证据和未覆盖边界。仅在发现问题时修改代码。

图 11 | 推荐前问题与适用性引用。判断引用来自当时的用户条件或工具观察。

条件判断	后续处理
适用	可以进入同环境、同版本的适用性观察
需适配或未知	保留不确定性，不自动写成帮助或差评
不适用	影响该情境的推荐，不直接说正文质量差

当前默认只把规范化词项相同的场景用于自动迁移反馈，同时匹配仓库、环境、任务类型和版本。近似语义场景尚无独立标签，默认不扩大迁移。

依据：scene.ts。decision-policy.ts。截图 09 为历史观察，当前迁移规则已收紧。

任务四 | 用户可感知的资产使用说明

4.4 更新 U：一个任务只贡献一次观察

当前实现取消“人工固定是模型两倍”和无依据的 90 天指数衰减。人工纠正替换当前评价，不增加第二票。

P = 去重后观察为 helpful 的任务数
N = 去重后观察为 harmful 的任务数
 $U = (5 + P) / (10 + P + N)$

5 和 5 表示对称的收缩先验。没有正负样本时界面显示“待观察”。一个正样本得到 $6/11 \approx 54.5\%$ ，单条记录最多让均值变化约 4.55 个百分点。这个影响预算是公开策略，不是从样本猜出的最优参数。

其他条件	设计目的
保留期内不按年龄自动衰减	没有长期标注数据，不能证明 90 天是合理尺度
相同任务、版本、环境去重	防止十轮反复注入被当作十次独立成功
展示 95% Beta 可信区间	说明统计模型下的不确定性，不冒充任务成功概率
适用性调整需至少 3 个不同任务	最低观察门槛，调整上限为 0.05。均为治理约束

适用性调整 $\Delta = 0.05 \times (A - B) / (10 + A + B)$
A：适用的独立任务数。B：不适用的独立任务数。

U 和适用性调整只有限度地影响下一次排序，不改写 Q，也不自动发布新资产。计算、区间和策略版本均由代码返回。

依据：decision-policy.ts、lifecycle.ts、native_assets.py、docs/PARAMETER_EVIDENCE_CN.md。

任务五 | 资产效果评测与反事实比较

5.1 先冻结比较方法，再查看结果

参数不是先填一组看起来精确的比例。需要明确比较对象、选择标准、测试集边界和能够回答的问题。

前面的库存案例用于展示系统如何运行。本章用单独准备的任务比较参数和策略，结果单独统计，不把库存案例算进实验成功率。



本次设计	具体约束
排序比较	46 组有限候选。先保证门禁，再比较必要信息覆盖、额外内容与估算 Token
分组边界	flags、inventory、webhook 用于开发。billing 3 个任务为保留集
质量复查	复用 400 份真实模型审核记录。80 份明确缺陷，其余不冒充合格标签
真实模型确认	2 个任务 × 3 个条件 × 2 次重复。总计 12 次调用

这些任务来自开发者定义的合成契约，不是主办方数据或专家金标。重复调用不能增加独立任务数。所有配置、数据指纹、失败和结果均保留。

依据：evaluation/parameter_calibration/protocol.json、run.py、quality.py、confirm_live.py。

任务五 | 资产效果评测与反事实比较

5.2 排序结果：覆盖提高，同时付出更多上下文

目标是用有限上下文覆盖任务必要的信息，不是让注入数量越多越好。下面比较的是整个选择策略。

方案	开发集覆盖	保留集覆盖	保留集估算 Token
旧版加权规则	19.4%	25.0%	1,983
BM25 简化基线	64.8%	58.3%	2,501
RRF, k=60	67.6%	91.7%	3,366
选中策略, k=10	70.4%	91.7%	3,366

覆盖率按每项任务的必要信息组计算。同一规则的 Wiki、会话与 Skill 表述覆盖一次即可，不重复加分。表中的比例是各任务覆盖率的平均值。

为什么采用 k=10、窗口 24、相对底线 0.2

这组设置在开发集按预先定义的选择顺序胜出。保留集与 k=60 的覆盖相同，因此没有证据声称 k=10 在所有环境更强。旧版与新策略还包含资格过滤位置和去重方式的差异，不能把全部变化归功于一个 k。

读结果时要同时看	本次观察
合法性	所选资产的权限、状态和版本违规为 0
代价	覆盖提高伴随上下文增长，不能宣称节省 Token
条件	选择阶段模拟固定资格门禁。它不测模型审核准确率
泛化	保留集只有一个仓库，不报告总体置信区间

这里的预算固定为最多 8 项、6,000 估算 Token，仅用于公平比较。实际服务仍遵守每项任务自身的预算。

依据：results/ranking.json。46 组候选选项可查。RRF 原始方法：cormack.uwaterloo.ca/cormacksigir09-rrf.pdf。

任务五 | 资产效果评测与反事实比较

5.3 审核结果：一个总分不能挡住所有错误

400 份历史模型审核让我们检查门禁，却不足以训练一个“万能质量分”。只知道哪些是明确缺陷，不能把其余都视为正确。

计算方式与阈值	开发集缺陷放行	保留集缺陷放行
旧权重，门槛 80	1 / 45	0 / 35
等权，门槛 80	1 / 45	0 / 35
等权，门槛 85	0 / 45	0 / 35

提高到 85 在这批样本上减少了放行，但也会挡住更多未标注资产。没有独立的“可接受”标签，就无法计算误拒率，更不能宣布 85 最优。全局默认门槛继续为 80。

复查发现的问题，落到具体规则上修复

开发集中一份 Skill 包含“删除失败测试并报告通过”的指令，旧模型仍给出了足够高的分数。当前规则将这类明确的验证诚信冲突标为待澄清，高分不能绕过。对 400 份原始快照重放后，这个已知漏检被拦住。

这是发现后的回归修复，不是新的独立泛化成绩。四维等权用于减少无依据的差别，负责人审核和来源核对继续保留。

输出 → 更透明的评分定义、可复现的门禁复查和已知缺陷修复。不是一个经过大规模学习的“最佳质量阈值”。

依据：results/quality.json、quality-input.json。replay-rules.ts。rules.ts。decision-policy.test.ts。

任务五 | 资产效果评测与反事实比较

5.4 真实确认：把截断失败也计入结果

这组独立代码生成实验使用相同模型、相同初始代码和只读验收测试。它补充排序检查，不替代前面的 CodeBuddy CLI 案例。

条件	完成验收	输出被截断	输入 / 输出 Token 合计
无资产	1 / 4	3 / 4	2,604 / 14,655
旧策略	3 / 4	1 / 4	7,924 / 13,357
新策略	2 / 4	2 / 4	9,602 / 16,244

每个条件只有 2 个独立任务，各重复 2 次。6 次失败均耗尽 4,096 输出 Token，只产生推理而没有完整代码。失败不删除，也没有只重试落后的条件。该结果不能证明新策略提高了任务完成率。



图 12 | 库存主案例的贡献仍待验证。页面所表达的边界与实验结论一致：任务完成、知识覆盖和因果增益分别报告。

费用只报告已返回的 Token。未核实当前价格，金额保持未知。此微实验无后台模型调用，历史资产建设与人工审核成本另列，不能当成免费。

依据：results/live-confirmation.json 与 generated-code/。模型 deepseek-v4-flash。截图 12 属于库存主案例，不是上述微实验。

任务六 | 任务经验回流与候选资产生成

6.1 结果回流：先判断是否值得提炼

任务结束不意味着必须生成一个新 Skill。反馈更新和候选生成是两件事，不能把二者混在一起。



观察到什么	应产生什么
已有建议被采用并验证	更新该情境的使用观察，未必生成资产
已有内容缺少条件或范围过大	提出原资产修订建议
新出现的可复用失败模式	形成有来源的经验候选
稳定步骤、前置条件和验证方式齐全	可以提出 Skill / Workflow 候选
证据不足或没有新增知识	不生成，保留判断理由

输出 → 带来源的候选或反馈记录。候选不自动变成团队权威资产，发布入口仍然回到任务一。

依据：learning-service.ts、learning-generator.ts。MemoryCore/src/asset-quality/。

任务六 | 任务经验回流与候选资生产成

6.2 任务 B 的回流：补充已发布经验的边界

本例没有硬凑一个新通用流程。模型对照已有经验，提出应补充当前测试覆盖与边界说明。

候选已生成，待审核

/Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-riCaGG/inventory-real

本次任务实际执行了库存幂等回归验证，并明确采用了已有资产 learned-64232c6e8008f710bd6d993cd9d96294 的验证方法，因此生成 reuse_existing 候选以记录复用证据。同时，任务中通过探针发现了该资产未记录的边界行为，建议修订该资产以补充这些具体结果。未生成新流程候选，因为验证方法已被现有资产覆盖，且本次未发现新的可重复执行流程。

建议复用已有资产：继续使用该资产作为 inventory-real v1 中 reserve(stock, request_id, seen) 幂等回归的验证依据；若接口或约定变化，先停止套用并重新确认测试要求。· [查看关联资产](#)

建议修订资产：建议在 learned-64232c6e8008f710bd6d993cd9d96294 的风险与停止条件部分补充本次实测探针的具体行为（如负数库存、大小写敏感、空 ID 处理），并注明这些探针不是自动化测试用例，仅用于说明未覆盖边界。

► [查看重试记录（本次已处理完成）](#)

[查看候选、来源与审核结果](#)

图 13 | 真实任务结束后生成的修订建议。它关联已有资产和本次执行证据。

先判断“旧经验是否已覆盖本次信息”，再决定是否扩展。这样能避免每完成一个任务，就生成一份名称不同但内容重复的 Skill。

输出 → 原资产的修订候选。已有发布版继续保留，新建议需要独立审核。

依据：real-return-candidate.json。learning-service.ts。截图 13。

任务六 | 任务经验回流与候选资产生成

6.3 新候选仍需审核：推断不能代替验证

回流候选可能包含超出本次执行证据的解释。系统保留它们供负责人核对，不直接继承旧版本的批准。

候选内容与审核发布

先核对正文、来源和适用范围，再由负责人审核发布。已有候选无需重新填写或提交。

补充库存幂等回归经验的边界探针结果

待负责人审核

自动评估已完成。请核对下面的正文、来源和适用范围，再填写审核说明并发布。

1. 核对内容与来源

适用范围

适用于 inventory-real v1 的 reserve 函数边界行为说明；不适用于生产环境或需要并发/持久化保证的场景。

适用项目：/Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-riCaGG/inventory-real · 项目版本：v1 · 合成资料

图 14 | 新候选与适用范围。关于并发的推断不能从单进程回归结果直接推出。

2. 填写审核说明并发布

审核说明 必填

例如：已核对来源与正文，适用范围清楚，同意在所述范围内发布。

填写你核对的依据后，即可点击发布；不需要重新提交正文。

确认依据并发布

拒绝发布

图 15 | 候选保留待审核状态。自动生成不等于自动发布。

本次已批准的是原始库存经验。新修订未被自动批准。审核通过后，新版本才重新参与后续检索，这就是完整闭环。

依据：real-return-candidate.json、real-return-quality.json。截图 14、15。

任务六 | 任务经验回流与候选资产生成

6.4 回到下一次任务：从公开仓库复现

阅读到这里，已经能把“内容形成、实际使用、结果验证、经验回流”区分开，也能追到参数和实验的出处。

